

AUTOMATION

Run Claude Code programmatically

 Copy page

Use the Agent SDK to run Claude Code programmatically from the CLI, Python, or TypeScript.

The **Agent SDK** gives you the same tools, agent loop, and context management that power Claude Code. It's available as a CLI for scripts and CI/CD, or as **Python** and **TypeScript** packages for full programmatic control.

To run Claude Code in non-interactive mode, pass `-p` with your prompt and the **CLI options** you need:

```
claude -p "Find and fix the bug in auth.py" --allowedTools "Read,Edit,Bash"
```

This page covers using the Agent SDK via the CLI (`claude -p`). For the Python and TypeScript SDK packages with structured outputs, tool approval callbacks, and native message objects, see the **full Agent SDK documentation**.

Basic usage

Add the `-p` (or `--print`) flag to any `claude` command to run it non-interactively. Not every **CLI option** combines with `-p` . Claude Code rejects `--bg` , and rejects `--cloud` with a task description, with an error naming the conflict; `--cloud` with a session ID and `-p` instead **queues a message into that cloud session** and exits. Options you'll combine with `-p` often include:

- `--continue` for **continuing conversations**

```
claude -p "What does the auth module do?"
```

Claude Code exits with code 0 on success and a non-zero code when the run fails, so your scripts can branch on the exit status. If you pass an invalid flag, Claude Code reports the error to stderr before the run starts. When a failure happens inside the run, such as missing authentication, Claude Code prints the failure as the result on stdout.

Start faster with bare mode

Add `--bare` to reduce startup time by skipping auto-discovery of hooks, skills, custom commands, subagents, plugins, MCP servers, auto memory, and CLAUDE.md. Without it, `claude -p` loads the same context an interactive session would, including anything configured in the working directory or `~/.claude`.

Bare mode is useful for CI and scripts where you need the same result on every machine. A hook in a teammate's `~/.claude` or an MCP server in the project's `.mcp.json` won't run, because bare mode never reads them. A directory you name with `--add-dir` is a partial exception: bare mode loads skills from its `.claude/skills/` folder, but still skips its `.claude/commands/` and `.claude/agents/` folders. Skills from additional directories covers what does and doesn't load.

Without `--bare`, a `-p` session runs the hooks in a project's `.claude/settings.json` and connects the servers in its `.mcp.json`, even in a folder you've never trusted. A `-p` session shows no workspace trust dialog and no per-server approval prompt. What runs before you trust a folder covers each kind of repository content under `-p` and how to keep it out.

This example runs a one-off summarize task in bare mode and pre-approves the Read tool so the call completes without a permission prompt. Set `ANTHROPIC_API_KEY` before running it, because bare mode doesn't use your subscription login:

the Anthropic API, set `ANTHROPIC_API_KEY` in the environment, with a key created in the **Claude Console**, or supply an `apiKeyHelper` in the `--settings` JSON. Amazon Bedrock, Google Cloud's Agent Platform, and Microsoft Foundry continue to read their own provider credentials as usual.

In bare mode Claude has access to the Bash, file read, and file edit tools. Pass any context you need with a flag:

To load	Use
System prompt additions	<code>--append-system-prompt</code> , <code>--append-system-prompt-file</code>
Settings	<code>--settings <file-or-json></code>
MCP servers	<code>--mcp-config <file-or-json></code>
Custom agents	<code>--agents <json></code>
A plugin	<code>--plugin-dir <path></code> , <code>--plugin-url <url></code>

❗ `--bare` is the recommended mode for scripted and SDK calls, and will become the default for `-p` in a future release.

Background tasks at exit

If Claude starts a **background Bash task** during a `claude -p` run, for example a dev server or a watch build, that shell is terminated about five seconds after Claude has returned its final result and stdin has closed. The grace period lets a task that finishes right after the result still deliver its output. Before v2.1.163, a never-exiting background process would hold the `claude -p` invocation open indefinitely.

Background **subagents** and workflows are exempt from the five-second grace because their result is part of the final output, so `claude -p` waits for them to complete. From v2.1.182, that wait is capped at ten minutes by default so a stuck background agent

If you stop a `claude -p` run with SIGTERM, for example with `kill` or from a process supervisor, Claude Code exits with code 143. Claude Code leaves the turn that was in progress unfinished and records no result for it. To end the turn instead, send SIGINT, or call the Agent SDK's `interrupt()`, before you stop the process.

On SIGTERM, Claude Code terminates the process tree of any Bash command that is still running. Claude Code then runs SessionEnd hooks and exits. While exiting, Claude Code starts no new tool call, sends no new model request, and runs no hook other than `SessionEnd`. If the run was in the middle of a command or waiting on a permission prompt when the signal arrived, Claude Code handles that step as follows:

- **Running a command:** Claude Code records the command as killed in the session.
- **Waiting for an answer to a permission prompt:** if you send SIGTERM to the process, Claude Code leaves the prompt unanswered. If your program closes the session through the Agent SDK, the SDK ends Claude Code's input before sending any signal, and Claude Code cancels the prompt as soon as the input ends.

When you resume the session, Claude Code continues the turn that SIGTERM left unfinished.

Examples

These examples highlight common CLI patterns. Where a command names a file such as `auth.py` or `build-error.txt`, substitute a file from your own project. In CI or other scripted environments, add `--bare` so Claude Code starts without loading the host's hooks, plugins, auto memory, or `CLAUDE.md`.

Pipe data through Claude

Non-interactive mode reads stdin, so you can pipe data in and redirect the response out like any other command-line tool.

With `--output-format json`, the response payload includes `total_cost_usd` and a per-model cost breakdown, so scripted callers can track spend per invocation without consulting the [usage dashboard](#). Both figures are client-side estimates and can differ from your actual bill.

❗ Piped stdin is capped at 10MB. If you exceed the cap, Claude Code exits with a clear error and a non-zero status. To work with larger inputs, write the content to a file and reference the file path in your prompt instead of piping it.

If Claude Code can't read stdin, for example because the process that started it disconnected its end, Claude Code prints a warning to stderr and continues with the prompt from the command line. Before v2.1.211, an unreadable stdin on Windows crashed the session or made it exit silently with no output.

Add Claude to a build script

You can wrap a non-interactive call in a script to use Claude as a project-specific linter or reviewer.

This `package.json` script pipes the diff against `main` into Claude and asks it to report typos. Piping the diff means Claude doesn't need Bash permission to read it, and the escaped double quotes keep the script portable to Windows:

```
{
  "scripts": {
    "lint:claude": "git diff main | claude -p \"you are a typo linter. for e"
  }
}
```

Run it with `npm run lint:claude`.

- `json` : structured JSON with result, session ID, and metadata
- `stream-json` : newline-delimited JSON for real-time streaming

This example returns a project summary as JSON with session metadata, with the text result in the `result` field:

```
claude -p "Summarize this project" --output-format json
```

To get output conforming to a specific schema, use `--output-format json` with `--json-schema` and a **JSON Schema** definition. The response includes metadata about the request (session ID, usage, etc.) with the structured output in the `structured_output` field.

This example extracts function names and returns them as an array of strings:

```
claude -p "Extract the main function names from auth.py" \  
--output-format json \  
--json-schema '{"type":"object","properties":{"functions":{"type":"array",
```

If the value isn't a valid JSON Schema, `claude` exits with `Error: --json-schema is not a valid JSON Schema` followed by the validator's diagnostic. Claude Code accepts schemas that use the `format` keyword, such as `"format": "email"`, but treats `format` as an annotation and doesn't enforce it. Before v2.1.205, Claude Code silently ignored an invalid schema and returned unstructured text, and treated any schema containing `format` as invalid.

```
# Extract structured output
claude -p "Extract function names from auth.py" \
  --output-format json \
  --json-schema '{"type":"object","properties":{"functions":{"type":
| jq '.structured_output'
```

Stream responses

Use `--output-format stream-json` with `--verbose` and `--include-partial-messages` to receive tokens as they're generated. Each line is a JSON object representing an event:

```
claude -p "Explain recursion" --output-format stream-json --verbose --include-
```

The last line of the stream is a `result` message with the final response text, cost, and session metadata.

If your consumer reads the stream slowly, Claude Code waits for the queued output to drain before exiting, scaling the wait with how much is still queued, capped at 30 seconds. Before v2.1.214 the exit wait was capped at about two seconds, which could cut off the end of a large response.

The following example uses `jq` to filter for text deltas and display just the streaming text. The `-r` flag outputs raw strings (no quotes) and `-j` joins without newlines so tokens stream continuously:

```
claude -p "Write a poem" --output-format stream-json --verbose --include-par
jq -r -j 'select(.type == "stream_event" and .event.delta.type? == "text_del'
```

Messages from **subagents** appear in the stream as `assistant` and `user` messages whose `parent_tool_use_id` field is the ID of the tool call that spawned the subagent. Messages from the main conversation carry `null` in that field.

By default, Claude Code emits only subagent `tool_use` and `tool_result` blocks. Pass `--forward-subagent-text` or set `CLAUDE_CODE_FORWARD_SUBAGENT_TEXT` to also emit subagent text and thinking blocks, so you can reconstruct each subagent's transcript. This requires Claude Code v2.1.211 or later.

When you enable either option, Claude Code forwards messages from **subagents at every nesting depth**: when a subagent spawns its own subagent, the nested subagent's messages carry the ID of the Agent tool call that spawned it in `parent_tool_use_id`, so you can rebuild the full nesting tree by following those IDs. Before v2.1.219, messages from nested subagents didn't appear in the stream.

Handle API retries

When an API request fails with a retryable error, Claude Code emits a `system/api_retry` event before retrying. You can use this to surface retry progress or implement custom backoff logic.

Field	Type	Description
<code>type</code>	<code>"system"</code>	message type
<code>subtype</code>	<code>"api_retry"</code>	identifies this as a retry event
<code>attempt</code>	integer	current attempt number, starting at 1
<code>max_retries</code>	integer	total retries permitted
<code>retry_delay_ms</code>	integer	milliseconds until the next attempt
<code>error_status</code>	integer or null	HTTP status code, or <code>null</code> for connection errors with no HTTP response

		<code>max_output_tokens</code> , or <code>unknown</code>
<code>uuid</code>	string	unique event identifier
<code>session_id</code>	string	session the event belongs to

Read session metadata

The `system/init` event reports session metadata including the model, tools, MCP servers, and loaded plugins. It is the first event in the stream unless startup events precede it:

- `plugin_install` events, when `CLAUDE_CODE_SYNC_PLUGIN_INSTALL` is set.
- `hook_started` , `hook_progress` , and `hook_response` **events**, while a configured `SessionStart` or `Setup` hook runs. These stream as the hook produces them. Claude Code v2.1.169 through v2.1.203 delivered them in one batch after the hook completed, still ahead of `system/init` ; v2.1.204 restored live delivery.

The event also carries an optional `capabilities` array of strings naming the protocol behaviors this Claude Code version implements, such as `interrupt_receipt_v1` or `interrupt_cancel_queued_v1` . Check it to feature-detect instead of comparing version strings, and ignore values you don't recognize. The field requires Claude Code v2.1.205 or later and is absent from earlier versions. See `SDKSystemMessage` for the capability list.

Fail CI when a plugin or MCP server doesn't load

Use the plugin fields in the `system/init` event to catch a plugin that didn't load:

Field	Type	Description
<code>plugins</code>	array	plugins that loaded successfully, each with <code>name</code> and <code>path</code>

demoted and absent from `plugins` . The key is omitted when there are no errors

Use the MCP server fields the same way. When you pass `--mcp-config` with `-p` , Claude Code waits for still-pending servers before running the first turn, up to the `MCP_TIMEOUT` startup timeout, 30 seconds by default. A remote server with a **cached tool list** skips the wait, shows `pending` in `system/init` , and connects on its first tool call. The wait requires Claude Code v2.1.221 or later.

Claude Code validates each `--mcp-config` entry at startup and skips entries that fail validation, for example a `url` entry with no `type` . The run continues and exits cleanly, so check these fields to catch a server that never loaded:

Field	Type	Description
<code>mcp_servers</code>	array	MCP servers in the session, each with <code>name</code> and <code>status</code>
<code>mcp_server_errors</code>	array	<code>--mcp-config</code> entries skipped by config validation, each with <code>name</code> , <code>type</code> , and <code>message</code> . <code>type</code> is a skip category such as <code>unknown_type</code> , <code>url_missing_type</code> , <code>invalid_config</code> , or <code>reserved_name</code> ; treat values you don't recognize as a generic skip. Affected servers are absent from <code>mcp_servers</code> . The key is omitted when there are no errors, so a CI gate can fail on a non-empty array. Requires Claude Code v2.1.219 or later

When you run the command by hand in a terminal, Claude Code also prints a startup warning to stderr, such as `Warning: 1 MCP server skipped due to invalid config:` , followed by the reason for each skipped entry. When you redirect stderr, or when a program such as a CI runner or an SDK host captures it, Claude Code prints no warning and reports the skipped entries only in the `mcp_server_errors` field. The warning requires Claude Code v2.1.219 or later.

Field	Type	Description
<code>type</code>	<code>"system"</code>	message type
<code>subtype</code>	<code>"plugin_install"</code>	identifies this as a plugin install event
<code>status</code>	<code>"started"</code> , <code>"installed"</code> , <code>"failed"</code> , or <code>"completed"</code>	<code>started</code> and <code>completed</code> bracket the overall install; <code>installed</code> and <code>failed</code> report individual marketplaces
<code>name</code>	string, optional	marketplace name, present on <code>installed</code> and <code>failed</code>
<code>error</code>	string, optional	failure message, present on <code>failed</code>
<code>uuid</code>	string	unique event identifier
<code>session_id</code>	string	session the event belongs to

Auto-approve tools

Use `--allowedTools` to let Claude use certain tools without prompting. This example runs a test suite and fixes failures, allowing Claude to execute Bash commands and read/edit files without asking for permission:

```
claude -p "Run the test suite and fix any failures" \
  --allowedTools "Bash,Read,Edit"
```

To set a baseline for the whole session instead of listing individual tools, pass a **permission mode**. For `-p`, the **built-in starting permission mode** is Manual on every plan, so pass the permission mode you want:

- `auto` : pass `--permission-mode auto` to have a classifier review most actions instead of you

- `acceptEdits` : Claude writes files without prompting, and Claude Code auto-approves common filesystem commands such as `mkdir` , `touch` , `mv` , and `cp` . The **actions no mode auto-approves** still apply. Apart from the read-only command set, other shell commands and network requests still need an `--allowedTools` entry or a `permissions.allow` rule. See **what `acceptEdits` auto-approves** for the full list

This example applies lint fixes with `acceptEdits` as the baseline:

```
claude -p "Apply the lint fixes" --permission-mode acceptEdits
```

Create a commit

This example reviews staged changes and creates a commit with an appropriate message:

```
claude -p "Look at my staged changes and create an appropriate commit" \  
  --allowedTools "Bash(git diff *),Bash(git log *),Bash(git status *),Bash(git
```

The `--allowedTools` flag uses **permission rule syntax**. The trailing `*` enables prefix matching, so `Bash(git diff *)` allows any command starting with `git diff` . The space before `*` is important: without it, `Bash(git diff*)` would also match `git diff-index` .

- ❗ User-invoked **skills** and custom commands work in `-p` mode: include `/skill-name` in the prompt string and Claude Code expands it before running. Built-in commands that only run in the terminal interface, such as `/login` , aren't available in `-p` mode. `/model` , `/effort` , `/fast` , `/color` , and `/rename` accept the value as an argument, for example `/model sonnet` , and `/mcp` with no argument prints a text summary of server status; these forms require Claude Code v2.1.205 or later and follow each command's **availability notes**. To change a setting from a `-p` invocation, pass `key=value` to `/config` , for example `/config thinking=false` .

```
gh pr diff "$1" | claude -p \  
  --append-system-prompt "You are a security engineer. Review for vulnerabil:  
  --output-format json
```

In the script, `"$1"` stands for the first argument you pass on the command line. Run `bash review.sh 123` and the shell replaces `"$1"` with `123`, so the script fetches the diff for PR 123. Claude Code prints the review as JSON, with the text in the `result` field.

See [system prompt flags](#) for more options including `--system-prompt` to fully replace the default prompt.

Continue conversations

Use `--continue` to continue the most recent conversation, or `--resume` with a session ID to continue a specific conversation. `--continue` skips background sessions. This example runs a review, then sends follow-up prompts:

```
# First request  
claude -p "Review this codebase for performance issues"  
  
# Continue the most recent conversation  
claude -p "Now focus on the database queries" --continue  
claude -p "Generate a summary of all issues found" --continue
```

If you're running multiple conversations, capture the session ID to resume a specific one:

```
session_id=$(claude -p "Start a review" --output-format json | jq -r '.session_id')  
claude -p "Continue that review" --resume "$session_id"
```

Next steps

- **Agent SDK quickstart**: build your first agent with Python or TypeScript
- **CLI reference**: all CLI flags and options
- **GitHub Actions**: use the Agent SDK in GitHub workflows
- **GitLab CI/CD**: use the Agent SDK in GitLab pipelines

Was this page helpful?

 Yes

 No



Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

[Trust center](#)

[Transparency](#)

Learn

[Courses](#)

[MCP connectors](#)

[Customer stories](#)

[Engineering blog](#)

[Events](#)

[Powered by Claude](#)

[Service partners](#)

[Startups program](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

Terms and policies

[Privacy choices](#)

[Privacy policy](#)

[Disclosure policy](#)

[Usage policy](#)

[Commercial terms](#)

[Consumer terms](#)